SYSTEM DESIGN

Chapter 59: Design a Search Autocomplete

Personal Notes

You start typing in Google's search box: "harr" — and instantly see "harry potter," "harrison ford," "harry styles." Type another letter: "harry" — the list updates. This chapter designs the system behind that experience. It's one of the most-asked system design interview problems, and it's beautiful because it sits precisely at the intersection of algorithms (tries from Ch 36!) and distributed systems.

The design has THREE core challenges: making prefix lookup fast enough to update on every keystroke (~50 ms end-to-end), ranking suggestions by relevance/popularity, and continuously ingesting new user queries to keep suggestions current. Along the way we'll build a distributed trie sharded across many nodes, add top-K ranking, design the update pipeline, handle personalization, and address fuzzy matching for typos. Beyond autocomplete, these patterns power every predictive text system on the internet.

1. The Problem

Design a search autocomplete service. Users type a prefix; the system returns 5-10 most-likely completions ranked by popularity, freshness, and (optionally) personalization. Suggestions update on every keystroke with imperceptible latency. Continuously learn from real user queries to keep suggestions current — trending queries should appear within minutes of surging.

```
User types "harr" in search box
   │
   ▼ (query fires on each keystroke)
Autocomplete service returns top 5 suggestions:
   - harry potter          (1.2B searches/year)
   - harrison ford         (200M/year)
   - harry styles          (180M/year)
   - harry and meghan      (45M/year)
   - harry connick jr      (12M/year)
   │
   ▼
User continues typing: "harry"
   Suggestions update — same top 5 minus "harrison ford"

Latency budget: <100 ms end-to-end for the whole cycle
```

2. Step 1 — Requirements Gathering

2.1 Functional Requirements

- Given a prefix, return top 5-10 completions

- Rankings driven by query popularity (how many people searched for it)
- Suggestions update in real-time as user types
- Continuously ingest new queries to update the ranking
- (Optional) Personalization based on user history
- (Optional) Typo tolerance / fuzzy matching
- (Optional) Filter offensive/inappropriate suggestions

2.2 Non-Functional Requirements

- Latency — server-side p99 < 20 ms, end-to-end < 100 ms
- Scale — Google Search: ~100K QPS globally; every user session fires many autocomplete requests
- Availability — 99.9%+ (if autocomplete is down, search still works, but UX degrades)
- Freshness — trending terms appear within ~1 hour of trending

2.3 Constraints to Confirm

- How many suggestions to return — 5 to 10 is standard
- Language + locale — global service, or English only?
- Max prefix length considered — usually caps around 20 chars (beyond that user is typing full query)
- Personalized or global rankings — personalization multiplies complexity

> **The critical clarification:** "When we say popularity — over what time window?" A word that was popular in 2015 ("Bing" — remember Bing?) shouldn't dominate today's suggestions. Ranking should heavily weight RECENT searches (last 24 hours + last 7 days) with some decay for older data. Confirm the recency policy upfront.

3. Step 2 — Capacity Estimation

Query Load

```
Assume ~10 billion searches/day → 10^10 / 86,400 ≈ 115K queries/sec

Each search generates ~10 autocomplete requests (one per keystroke)
→ ~1.15M autocomplete requests/sec

Peak: ~3M autocomplete requests/sec
This is the READ QPS the system must serve.
```

Storage — Trie Size

```
Unique queries stored: ~1B (most-common tail is huge)
Avg query length: 20 chars
Storage per node:
```

```
      26 pointers × 8 bytes = 208 B
      + frequency counter (8 B)
      + top-K list (5 strings × ~30 B ≈ 150 B)
   Total: ~400 B per trie node

   Nodes for 1B queries × 20 chars avg = 2×10^10 nodes (naïve)
   With prefix sharing, actual ~2×10^9 nodes → 800 GB

   Sharded across ~50 nodes → 16 GB per node in RAM.
```

Ingest Load

```
   115K searches/sec globally that we can learn from
   Each search updates counters along the trie path of ~20 nodes
   → ~2.3M counter updates/sec

   Must NOT do this synchronously — batch via Kafka.
```

4. Step 3 — API Design

```
GET /autocomplete?q=harr&limit=10&locale=en_US

Response:
{
  "prefix": "harr",
  "suggestions": [
    { "text": "harry potter",     "score": 1.2e9, "type": "popular" },
    { "text": "harrison ford",     "score": 2.0e8, "type": "popular" },
    { "text": "harry styles",      "score": 1.8e8, "type": "trending" },
    { "text": "harry and meghan",  "score": 4.5e7, "type": "popular" },
    { "text": "harry connick jr",  "score": 1.2e7, "type": "popular" }
  ],
  "response_time_ms": 8
}

POST /log-search
{
  "query": "harry styles concert",
  "user_id": "abc",   // optional
  "locale": "en_US"
}
→ 202 Accepted   (fire-and-forget event ingestion)
```

GET should be idempotent + cache-friendly: The autocomplete GET endpoint is called on every keystroke, so it MUST be safe to cache aggressively (client-side, CDN, backend). Include locale + a compact query as the cache key. The write-side (logging searches for learning) is a separate endpoint that's write-only. Never combine them.

5. Step 4 — The Core Algorithm (Trie Recap from Ch 36)

A TRIE is a tree where each node represents a character, and each root-to-node path represents a prefix. Perfect data structure for prefix-based lookup — O(L) to find all completions of a prefix of length L, regardless of the total number of queries stored.

```
     Trie for words: "cat", "cats", "car", "care", "career"

                    (root)
                      |
                      c
                      |
                      a
                     / \
                    t   r
                   /   /|\
                  s   * e (word "car")
                (word) |  (word "care")
                       e
                       |
                       r
                       |  (word "career")

     * = end-of-word marker
     Prefix "car" walks c→a→r → 4 completions descend from here
```

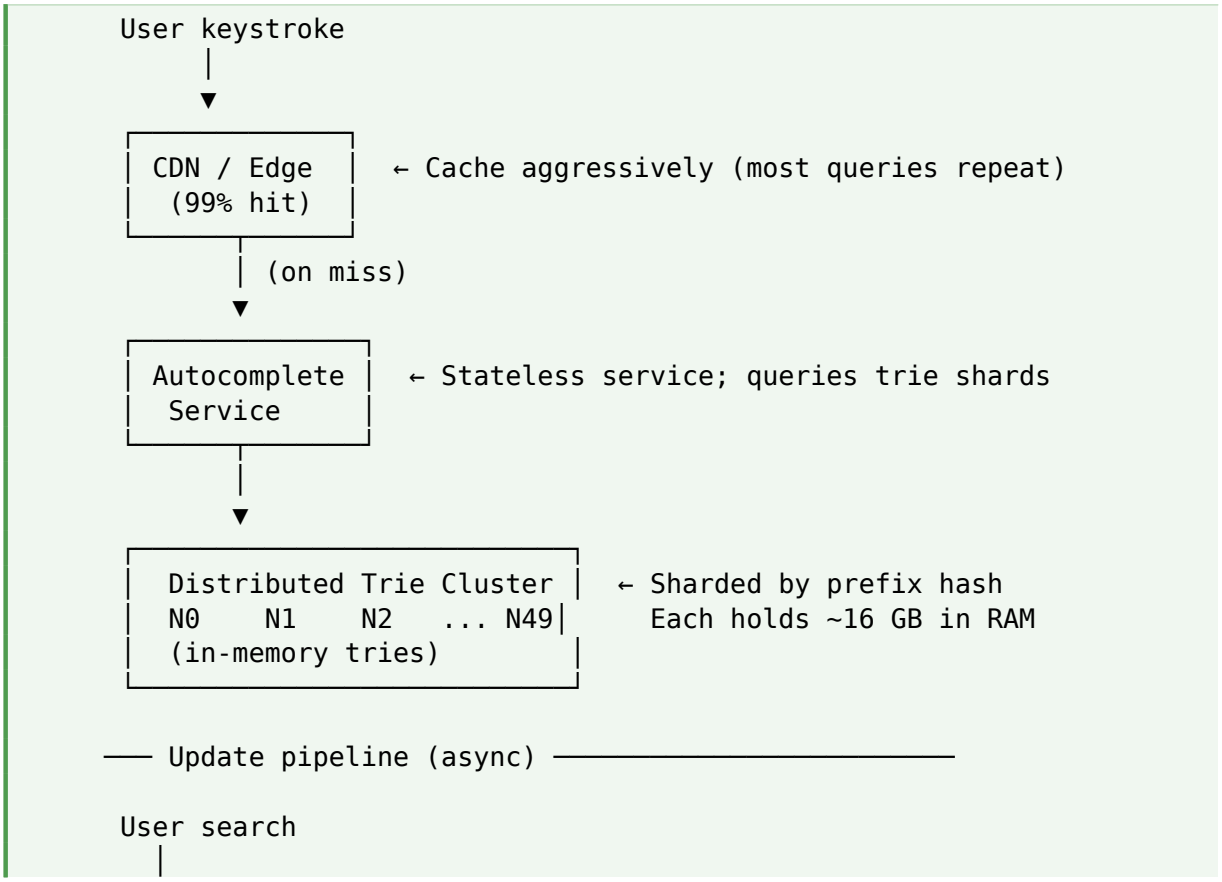
Trie for Autocomplete — With Popularity

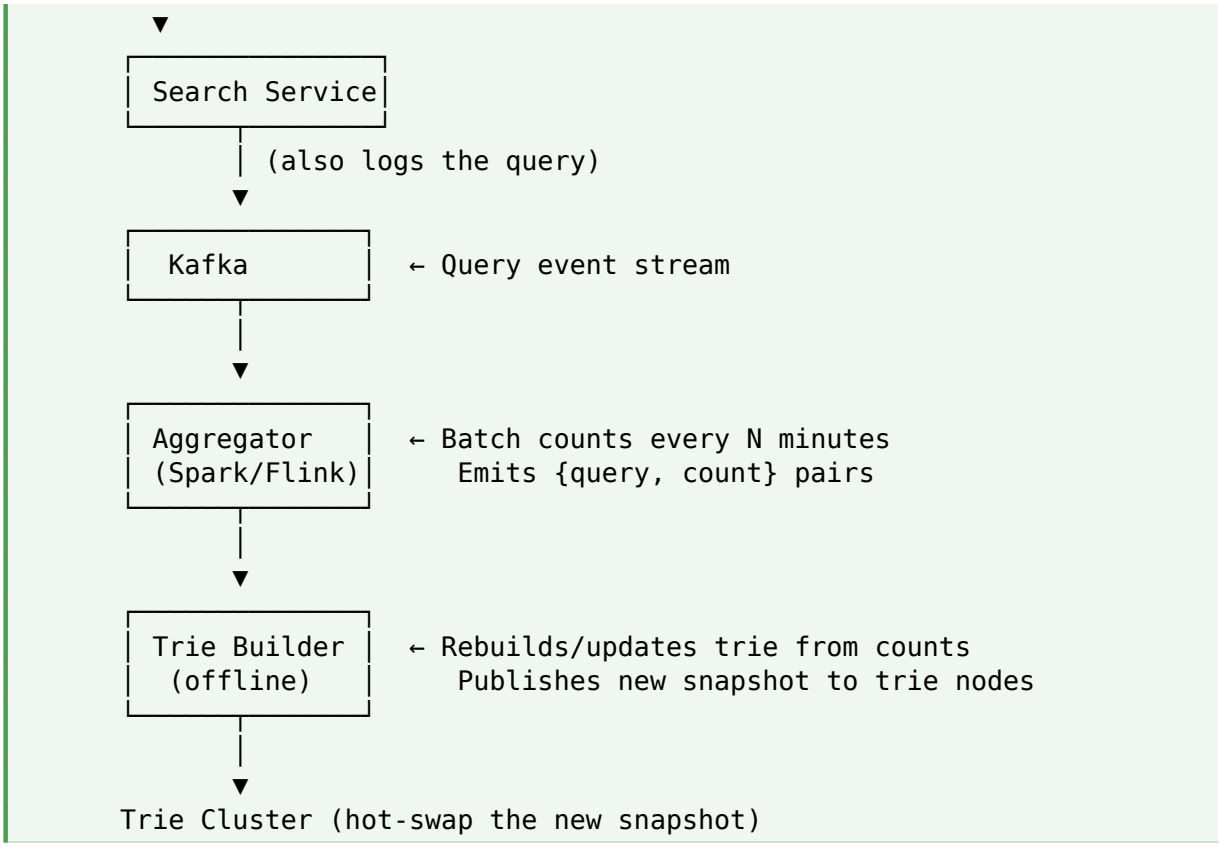
```
class TrieNode {
    Map<Character, TrieNode> children = new HashMap<>();
    boolean isEndOfWord;
    long frequency;                          // total searches for this
exact string
    List<Suggestion> topK;                    // pre-computed top-K
completions
                                              // for prefixes ending at
this node
}

class Suggestion {
    String text;
    long score;
}

// Insertion of "harry potter" with frequency 1B:
public void insert(String query, long count) {
    TrieNode cur = root;
    for (char c : query.toCharArray()) {
        cur.children.putIfAbsent(c, new TrieNode());
```

```
        cur = cur.children.get(c);
    }
    cur.isEndOfWord = true;
    cur.frequency += count;
}

// Retrieval — key trick: read pre-computed topK at prefix node
public List<Suggestion> autocomplete(String prefix) {
    TrieNode cur = root;
    for (char c : prefix.toCharArray()) {
        cur = cur.children.get(c);
        if (cur == null) return Collections.emptyList();
    }
    return cur.topK;    // O(1) — pre-computed
}
```

The pre-computed top-K trick: A naive trie walks all descendants to find completions on every query — millions of comparisons for popular prefixes like "a" or "the." Real autocomplete systems PRECOMPUTE top-K completions at each node during offline builds. Query becomes O(L) trie descent + O(1) result. Latency drops from ~100 ms to <5 ms.

6. Step 5 — High-Level Architecture





Two Parallel Systems

- READ PATH — hot, sub-10 ms latency, serves ~3M QPS: CDN → service → trie cluster
- WRITE PATH — batch, tolerates hours of latency: Kafka → aggregator → trie builder → deploy new snapshot

> **Never do trie updates on the hot read path:** The temptation is "user searches → increment counter in real-time trie." This would require synchronous writes to shared state on every search — killing latency and throughput. Real systems batch: log to Kafka, aggregate offline, rebuild trie periodically. The 1-hour lag in trending updates is acceptable in exchange for read-side speed.

7. Deep Dive: Sharding the Trie

One node can't hold 800 GB of trie in RAM. Shard across ~50 nodes. But how? The shard key determines everything.

Bad Approach — Shard by First Character

```
26 shards, one per letter:
   Shard "a" gets all queries starting with a
   Shard "b" gets all starting with b
   ...
```

```
Problem: skew.
   "s" has huge volume (search, spotify, ...)
   "z" is tiny.
One shard overloaded, one idle. Doesn't scale evenly.
```

Better — Shard by Hash of Prefix

```
For each 2-char (or 3-char) prefix, hash to a shard.
"ha" → shard 12
"th" → shard 3
"gh" → shard 44
...

Even distribution. All completions of "ha" live on shard 12.

For query "harr":
   Route to shard(hash("ha")) → shard 12
   That shard has full trie for the "ha" subtree.
   Descend the trie locally to return top-K.
```

Even Better — Distributed by Trie Subtree

For very popular first characters ("a", "s"), further split those subtrees across multiple nodes. This is fractal sharding — the trie splits naturally by structure. Each shard holds ~16 GB and serves queries for its allocated prefixes.

> **The rebalancing problem:** If shard 12 holds "ha" queries and they become viral overnight, that shard's load spikes 10×. Solution: hot-key detection + on-the-fly resharding. But it's complex. Interview-level answer: mention rebalancing as a known challenge; skip full solution unless pushed.

8. Deep Dive: The Ranking Function

"Top-K" isn't just "most searched." A great autocomplete blends multiple signals. Even a simple linear model beats pure frequency.

```
score(query) =
    W1 * log(all_time_count)                    // popularity
  + W2 * recent_count_24h                        // recency
  + W3 * recent_count_7d                         // trending signal
  + W4 * click_through_rate                      // are people actually
happy?
  - W5 * offensiveness_score                     // safety
  + W6 * personalization_bonus(query, user)      // per-user

// Weights tuned via A/B testing.
// Real Google uses ML models with 100+ signals, but the shape is the
same.
```

Handling Trending Queries

- Compute a SEPARATE trending index from just the last N hours
- At query time, boost trending completions by W3 (adjustable)
- Trending events (breaking news, viral moments) surface within minutes

Offensiveness / Safety

- Blocklist known offensive queries — never suggest them
- Auto-detect emerging patterns — new offensive queries appearing rapidly
- Manual review pipeline for edge cases (legal, medical, self-harm)
- Different rules per locale — jurisdiction-specific content restrictions

9. Deep Dive: The Update Pipeline

How does "harry styles concert" go from a user typing it once to appearing in autocomplete? The pipeline has four stages, each running at different frequencies.

1. INGEST — Every search logs a {query, timestamp, user_id, locale} event to Kafka (~115K events/sec)
2. AGGREGATE — Spark/Flink job consumes Kafka, computes counts per (query, time_bucket) — runs every 10-30 minutes
3. MERGE — Aggregated counts merged into a rolling window: last hour, last day, last week, all-time
4. REBUILD — Trie builder consumes rolling counts, updates or rebuilds trie snapshots — runs hourly

Snapshot Deployment

```
Old trie snapshot v42 running on shard 12.
New snapshot v43 ready to deploy.

Approach 1 — Blue/Green:
   Load v43 on separate memory region on same node.
   Once loaded, atomically switch pointer.
   Free v42.
   Cost: 2× memory during swap.

Approach 2 — Rolling Deploy:
   Take one shard replica offline at a time.
   Load new snapshot, put back into service.
   Continue for remaining replicas.
   Cost: temporarily reduced capacity per shard.

Approach 3 — Delta Updates:
   Ship only the CHANGES since last snapshot.
   Trie applies updates in-place.
```

```
   Cost: complex; risk of gradual drift.

Interview default: Blue/Green — simple, safe.
```

> **Rebuilding is expensive:** Rebuilding an 800 GB trie from scratch takes hours. Instead, most systems rebuild PER SHARD independently. Each shard rebuilds every N hours; the global system stays fresh. If one shard's rebuild fails, others continue serving. This is a classic "parallelize offline batch work" pattern.

10. Deep Dive: Caching

Client-Side Cache

- Browser or app caches responses per (prefix, locale)
- Typical user types slowly enough that repeated prefixes cache-hit
- Short TTL (~5 min) — trending changes should propagate

CDN Cache

- Cache the JSON response at CDN edge — the same "harr" query returns the same top-K globally (mostly)
- ~99% cache hit rate expected for the head of the distribution
- Personalized queries bypass CDN (or use a personalization suffix in cache key)

Server-Side Cache

- In-memory LRU on autocomplete service — top 100K prefixes covered
- Fallback path for CDN misses

```
Cache hierarchy:

   Client   → 30% hit rate (per user)
   CDN edge → 95% of remaining
   Service  → 99% of remaining
   Trie     → 0.01% actually reach here

End-user perceived latency: <20 ms for the vast majority.
Trie only really works when serving the LONG TAIL of unique queries.
```

11. Deep Dive: Personalization

Should "john" suggest "John Cena" or "John Mayer" first? Depends on the user. Personalization blends global rankings with per-user signals.

Approach — Rescoring at Query Time

5. Fetch top-K global suggestions from the trie (fast, cacheable)

6. Fetch user's history/preferences from a per-user profile store
7. RESCORE the K global candidates using user signals
8. Return top 5 after rescore

```
List<Suggestion> personalized(String prefix, User user) {
    List<Suggestion> globals = trieCluster.topK(prefix, K=20);
    return globals.stream()
        .map(s -> new Suggestion(s.text, rescore(s, user)))
        .sorted(byScore)
        .limit(5)
        .collect(toList());
}

double rescore(Suggestion s, User user) {
    double score = s.baseScore;
    if (user.hasHistoryWith(s.text)) score *= 5;
    if (user.locale.matches(s.locale)) score *= 2;
    if (user.recentContext.overlaps(s.text)) score *= 1.5;
    return score;
}
```

Fetch more, filter down: The pattern is: fetch top-20 (or top-50) from the global trie, THEN personalize. Fetching more candidates than needed lets the personalization layer have material to work with. Fetching only top-5 globally would give the personalizer nothing to reorder — the user might have already discarded their preferred candidate.

12. Deep Dive: Typo Tolerance / Fuzzy Matching

User types "harri" (typo for "harry"). Naive trie descent finds nothing — "harri" has no completions. Best-in-class autocomplete gracefully handles typos and gives "harry potter" anyway.

Approaches

Approach	Method	Cost
Edit distance search	For each prefix, try all edits within Damerau-Levenshtein 1	O(L × alphabet × 4) per query — expensive
Bigram indexing	Index queries by 2-char shingles; match candidates by shared bigrams	Broader recall, still needs ranking
Phonetic hashing	Metaphone/Soundex — match by pronunciation, not spelling	Great for name typos
ML re-ranking	Learn common typo patterns from user click data	Requires training data + model

Interview-Level Answer

"For fuzzy matching, I'd generate variants of the input within edit distance 1 (deletions, transpositions, replacements) and look up each variant in the trie. Limit to edit distance 1 for latency. For rich systems, integrate an ML re-ranker that learns typo patterns from click-through data — Google Search does this at massive scale."

13. Additional Features

13.1 Query Suggestions Beyond Words

Google shows more than just query completions — it shows PLACES ("harry's bar near me"), PEOPLE ("harry potter"), PRODUCTS. Each source is a separate index; the query fires against multiple in parallel; results are BLENDED and re-ranked.

13.2 Multi-Language Support

One trie per (locale, language). Users get suggestions in their language. For multilingual users, blend results from top 2 languages. Locale-specific trending signals — during World Cup, football queries dominate in Brazil but not in Japan.

13.3 Session Context

User just searched "iPhone 15 review." Types "b" — should suggest "best iphone 15 case," not "bank of america." Session-aware ranking boosts candidates related to recent queries in the same session.

13.4 A/B Testing Framework

Which ranking function is best? Serve variant A to 50% of users, variant B to the other 50%. Track engagement (click-through rate, session duration). Roll out the winner. This continuous experimentation is why Google Search feels progressively smarter.

14. Failure Modes

Failure	Mitigation
Trie shard down	Consistent hashing routes to next shard; that shard doesn't have the data → degraded results
Trie shard slow	Circuit breaker → return cached results; log alert
Kafka backlog builds up	Trending updates lag; static rankings still work
Rebuild pipeline broken	Trie ages; suggestions still work but freshness suffers
Autocomplete service down	Search still works — no suggestions displayed (graceful degradation)

Failure	Mitigation
Cache poisoning (bad suggestion)	Purge affected keys from CDN + local caches; manual review
Trending brigading (astroturf)	Deduplicate by IP+device; require geographic spread for trending status

> **Autocomplete is a nice-to-have:** Unlike core search, autocomplete failing gracefully degrades — the user just types their full query. This means the availability requirement is 99.9%, not 99.999%. In your design, mention this — it changes trade-offs (can go simpler on replication, skip cross-region for MVP).

15. Trade-offs Summary

Decision	Trade-off
Trie vs inverted index	Trie = O(L) prefix lookup, in-memory; inverted index = disk-friendly but slower prefix ops
Pre-compute top-K vs runtime	Pre-compute = fast reads, large storage; runtime = slower, smaller trie
Real-time vs batch trending	Real-time = fresh but expensive coordination; batch = ~1 hour lag but simple
Personalization at query time vs offline	Query-time = fresh signals; offline precompute = fast but stale
Global rankings vs per-locale	Global = simple; per-locale = culturally relevant, more storage
Fuzzy matching yes/no	Yes = handles typos, ~2× compute; no = simpler + faster
Shard by prefix vs subtree	Prefix hash = uniform; subtree = follows natural distribution

16. Common Mistakes

Mistake 1: Not pre-computing top-K

A naive trie walks all descendants of the prefix node on every query — millions of comparisons for hot prefixes. Store precomputed top-K completions AT each node during offline build. Reads become O(L) + O(K), independent of subtree size.

Mistake 2: Real-time trie updates on the hot path

Trying to increment counters synchronously on every search would kill throughput. Always batch: log to Kafka, aggregate offline, rebuild snapshots periodically. Accept 1-hour lag for trending updates.

Mistake 3: Sharding by first character

Distribution is heavily skewed — 's' has 10× the volume of 'z'. Hash-based prefix sharding gives uniform load. Bring this up unprompted — interviewers often set this trap.

Mistake 4: Ignoring caching

The distribution is Pareto — a small number of prefixes account for most queries. Aggressive multi-layer caching (client + CDN + service) means the trie only serves the tail. Missing this makes the design over-provisioned by 100×.

Mistake 5: One-size-fits-all ranking

Trending queries need different treatment than long-tail queries. Personalized users need different treatment than anonymous ones. Design the ranking as a MODEL with pluggable signals, not a fixed formula.

Mistake 6: No safety/moderation

Autocomplete has serious real-world impact — it can promote hate speech, misinformation, or dangerous queries. Blocklists + auto-detection are non-negotiable. Bring this up — it shows product maturity.

Mistake 7: Forgetting graceful degradation

If autocomplete is down, search still works — the user just types their full query. Don't over-engineer availability. 99.9% is fine; you don't need 99.999% like the search service itself.

17. Best Practices

- Use TRIES for prefix lookup — O(L) descent regardless of database size
- Pre-compute TOP-K at each trie node during offline builds — reads become O(L)
- Shard the trie by 2-3 char prefix HASH — uniform distribution
- Update pipeline is ASYNC: Kafka → aggregate → rebuild trie hourly
- Multi-layer caching: client → CDN → service → trie — 99%+ hits above the trie
- Ranking blends popularity + recency + trending + personalization signals
- Fetch top-K, personalize LOCALLY — reorder within a small candidate set
- Blue/green snapshot deploys — safest way to swap tries
- Blocklist + auto-detection for offensive queries — safety-first
- Graceful degradation — if autocomplete is down, search still works

18. Quick Reference

Concept	Meaning
Trie	Tree data structure for prefix lookup — O(L) per query
Prefix	Substring at the start of a word — what user typed so far
Top-K	K most relevant completions for a prefix
Pre-compute top-K	Store top-K at each trie node during offline build
Sharding by prefix hash	Hash 2-3 char prefix → shard; uniform distribution
Async update pipeline	Kafka → aggregate → rebuild trie snapshot hourly
Blue/green deploy	Load new snapshot, atomically swap pointer
Personalization	Rescore top-N global candidates using user signals
Trending	Boost queries with recent surge in count
Fuzzy matching	Generate edit-distance-1 variants; lookup each
Cache hierarchy	Client → CDN → service → trie; only tail hits trie
Ranking function	Weighted blend: popularity + recency + trending + CTR
Graceful degradation	Autocomplete failure ≠ search failure

19. Related Interview Problems

Autocomplete patterns generalize to many other systems:

9. Design a Search Engine — autocomplete is one component; also need indexer + ranker + query engine
10. Design Amazon's Product Autocomplete — trie + attributes (brand, category, price)
11. Design Twitter Search Autocomplete — heavy on people (@handles) and topics (#hashtags)
12. Design Code Autocomplete (IDE) — token-based, context-aware, local trie
13. Design Email Address Suggestions — smaller domain, personal contacts prioritized
14. Design an Address Autocomplete (Google Maps) — geographic proximity signals dominate
15. Design a Chat Emoji Picker — trie over emoji names + shortcuts
16. Design a Command Palette (VS Code, Slack) — fuzzy matching over available actions

20. Final Takeaway

Search autocomplete is a beautiful integration of DSA and distributed systems. The core data structure is one you built in Ch 36 — a trie. Wrap it with (1) pre-computed top-K at each node for O(L) reads, (2) sharding by prefix hash for horizontal scale, (3) an async ingest pipeline for freshness, and (4) aggressive caching so the trie only serves the tail. Add personalization + safety on top. That's essentially Google's actual design at a conceptual level.

For interviews, walk it top-down: trie with pre-computed top-K (algorithm) → sharding by prefix hash (systems) → async update pipeline via Kafka (data engineering) → multi-layer caching (performance) → personalization/safety (product). Then be ready for deep-dives: "How would you rebuild the trie? What about typos? What if a shard gets hot?" These questions probe how well you understand the interplay of algorithms and infrastructure.

> **Key Takeaway:** Autocomplete = TRIE + TOP-K precomputation + ASYNC INGEST + CACHING. Store PRECOMPUTED top-K at each trie node → O(L) reads regardless of subtree size. SHARD by prefix hash (not by first char — skewed). Update via ASYNC pipeline: Kafka → Spark/Flink aggregate → rebuild trie snapshot HOURLY. Deploy snapshots BLUE/GREEN. CACHE HIERARCHY (client → CDN → service → trie) means 99%+ of queries never hit trie. RANKING blends popularity + recency + trending + CTR + personalization. Fetch top-20 globally, personalize LOCALLY to top-5. Fuzzy matching via edit-distance-1 variants. GRACEFUL DEGRADATION — autocomplete down ≠ search down.

21. What's Next?

Autocomplete done. Remaining designs and deep dives:

- Design a Search Engine — the full stack: crawler (Ch 58) + indexer + ranker + query serving
- Design a Notification System — cross-channel delivery, priorities, batching, dedup
- Design a Payment System — idempotency, sagas, PCI compliance, financial invariants
- Design a Metrics/Monitoring System (Datadog-style) — time-series, retention tiers
- Design a Distributed Job Scheduler — Cron at scale, resilient task queues
- Deep Dive: Databases Internals — B-trees, LSM trees, WAL, MVCC
- Deep Dive: Kafka Architecture — partitions, consumer groups, exactly-once semantics
- Deep Dive: Consensus Algorithms — Raft (worked example), Paxos, leader election
- Deep Dive: Distributed Transactions — 2PC, sagas, event sourcing